Technical Documentation

Rental Long-Stay Prediction & Feature Analysis

Task 7 - IUM 2026L Project

Nocarz Rental Platform

Authors:
Tomasz Okoń, Andrii-Stepan Pryimak

May 31, 2026

Contents

1. Introduction and Problem Definition

1.1 Business Context

The objective of Task 7 for the Nocarz portal was formulated directly from a core business challenge: *"We do not fully understand the criteria used by customers who book longer stays."* Uncovering these criteria is critical for optimizing marketing strategies and enabling customer service consultants to personalize offers. Long-term bookings are highly desirable as they provide guaranteed revenue streams and significantly reduce operational overhead, such as cleaning and check-in logistics.

1.2 Machine Learning Task Definition

We translated this vague business inquiry into a rigorous **Binary Classification** problem combined with advanced **Explainable AI (XAI)**.

- **Target Variable (`is_long_stay`):** Derived by calculating the duration between `booking_date` and the actual stay length. Stays lasting 7 days or more were labeled as 1 (Positive Class), while shorter stays were labeled as 0.
- **Success Criteria:** Achieving an Area Under the ROC Curve (AUC) significantly higher than a prior-based dummy baseline, and successfully deploying a microservice capable of serving real-time predictions alongside local SHAP explanations.

2. Exploratory Data Analysis (EDA)

The foundation of a robust predictive model is a deep understanding of the underlying data. The raw dataset consisted of 5 distinct relational tables: listings, user sessions, users, reviews, and calendar data. Notebook `1_eda.ipynb` was utilized to verify data quality, completeness, and initial correlations.

2.1 Continuous Features Analysis

Continuous variables such as base price (`price_num`), review ratings (`review_scores_rating`), and the total number of reviews strongly influence consumer decisions.

Table 2.1: Descriptive Statistics for Key Continuous Features

Metric	Price (USD)	Rating Score	Total Reviews
Count	199,306	228,021	228,021
Mean	131.74	4.77	257.36
Std. Dev.	290.83	0.18	263.01
Min	5.00	1.00	1.00
Median (50%)	109.00	4.81	189.00
Max	50,000.00	5.00	1,628.00

The distributions exhibit characteristics typical of the real estate and rental markets: a pronounced long-tail distribution for prices (driven by luxury outliers) and a strong left-skew for ratings, indicating that the vast majority of properties are highly rated.

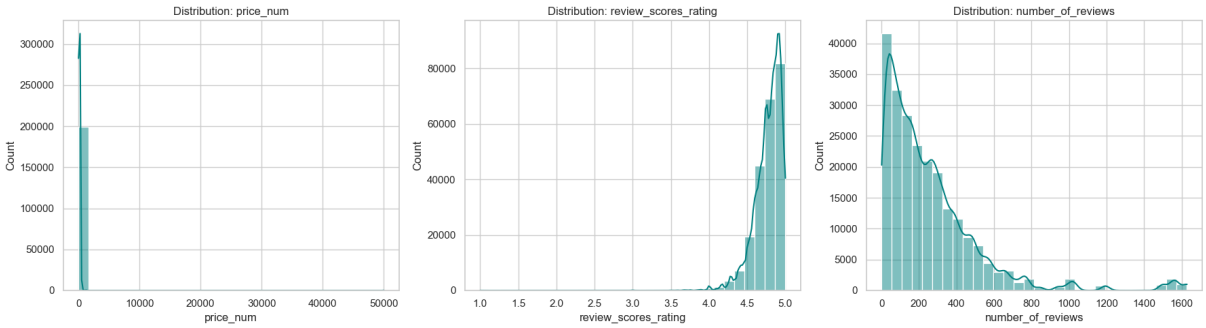

Figure 2.1: Output from `1_eda.ipynb`: Distributions of continuous features.

2.2 Discrete Features Analysis

Capacity-related attributes dictate the demographic a listing attracts. The data shows that the dominant property type is a 1-bedroom apartment accommodating 2 to 4 guests.

Table 2.2: Descriptive Statistics for Discrete Features (Capacity)

Metric	Accommodates (Guests)	Beds	Bedrooms
Mean	3.53	2.30	1.34
Std. Dev.	2.24	1.73	0.88
Min	1.00	0.00	0.00
Median (50%)	3.00	2.00	1.00
Max	16.00	15.00	14.00

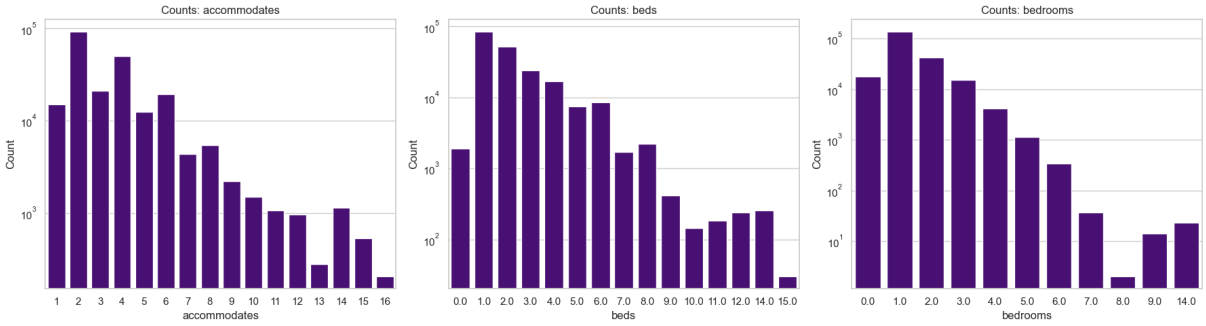

Figure 2.2: Output from `1_eda.ipynb`: Count plots for capacity features.

3. Feature Engineering and Preprocessing

The `2_data_prep.ipynb` notebook was responsible for aggregating the raw relational schemas into a single flat modeling dataframe.

3.1 Feature Extraction and Aggregation

Key engineering steps included:

1. **Calendar Analysis:** Transforming daily booking records into an aggregated `availability_rate` per listing.
2. **Text Mining (Amenities):** The unstructured `amenities` text column was parsed into binary flags (e.g., `has_kitchen`, `has_washer`, `has_workspace`). Furthermore, an aggregate `amenities_count` was calculated under the hypothesis that fully-equipped properties attract longer stays.
3. **Temporal Features:** We computed `lead_time_days`, representing the number of days between the booking creation date and the actual check-in date.

3.2 Data Cleaning and Imputation

Categorical variables (e.g., `room_type`) were transformed using One-Hot Encoding. Missing numerical values were addressed systematically: missing review scores were replaced with the median, missing review text lengths were zeroed, and missing calendar availability was assumed to be a neutral 50%. The finalized, model-ready dataset was exported to `processed_data.csv`.

4. Model Architecture and Evaluation

Modeling experiments were conducted in `3_training.ipynb`. The dataset (comprising over 220,000 records) was split into training (80%) and holdout test (20%) sets, using stratified sampling to preserve the target class distribution.

4.1 Baseline and Advanced Models

- **Baseline Model:** `DummyClassifier(strategy='prior')`. This classifier blindly predicts the majority class distribution observed in the training set, serving as a sanity check.
- **Advanced Model (XGBoost):** A robust Gradient Boosting Decision Tree architecture was selected. To prevent overfitting, the following hyperparameters were applied: `n_estimators=200`, `max_depth=6`, and `learning_rate=0.05`.

4.2 Performance Metrics

The XGBoost model demonstrated a radical improvement over the baseline, elevating the **AUC from a random 0.5000 to an outstanding 0.9635**.

Using a standard 0.5 decision threshold, the classification report yielded the following results, demonstrating the model's high precision and recall for both classes.

Table 4.1: Classification Report for the XGBoost Model

Class	Precision	Recall	F1-Score	Support
0.0 (Short Stay)	0.95	0.92	0.94	30,650
1.0 (Long Stay)	0.85	0.91	0.88	14,955
Accuracy		**0.92**		45,605
Macro Avg	0.90	0.91	0.91	45,605
Weighted Avg	0.92	0.92	0.92	45,605

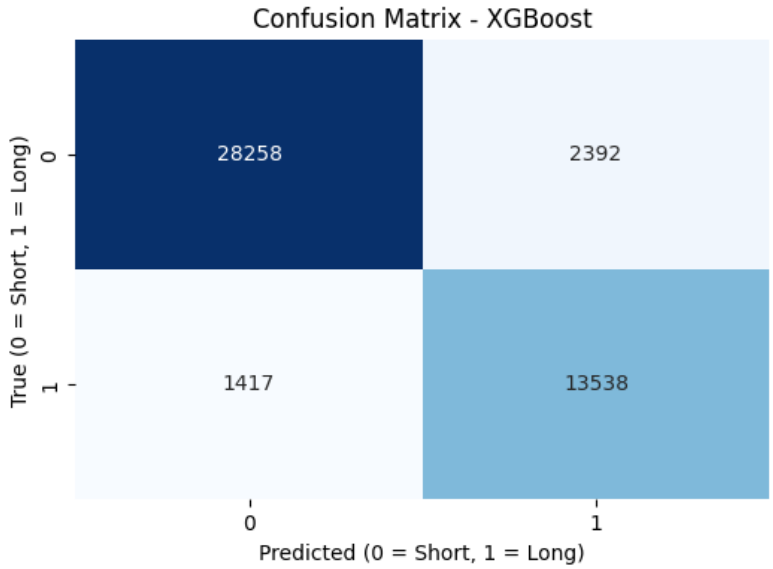

Figure 4.1: Confusion Matrix for the XGBoost Model.

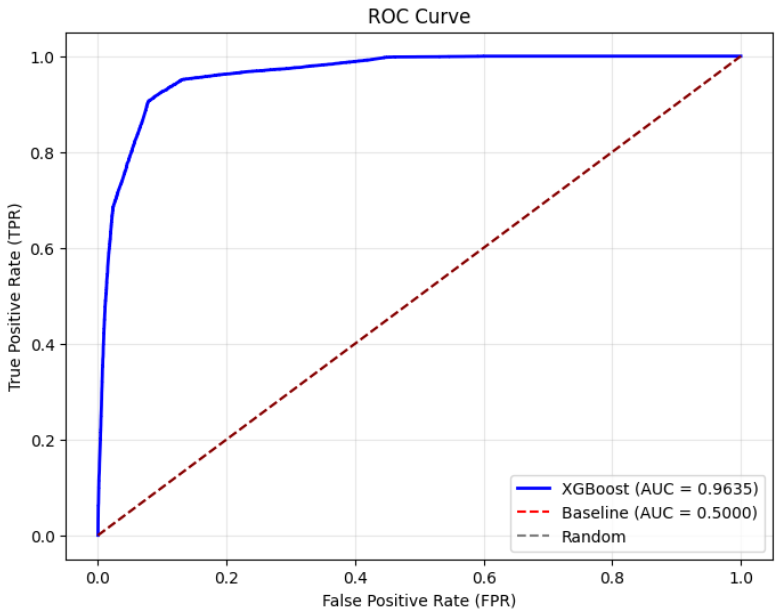

Figure 4.2: ROC Curve illustrating the massive predictive advantage of XGBoost over the Baseline.

5. Model Explainability (SHAP Analysis)

To satisfy the core business mandate of understanding *why* customers make specific booking decisions, we integrated SHAP (SHapley Additive exPlanations) into the pipeline. An in-depth analysis was documented in `6_shap.ipynb`.

5.1 Global Feature Importance

Summary Bar Plots revealed the most critical attributes driving the model's decisions. The top factors were: `lead_time_days`, `amenities_count`, followed closely by pricing and shared-room indicators.

However, the **Beeswarm plot** provided the most actionable business intelligence by displaying the *direction* of impact. For example, high values of `lead_time_days` (indicated by red dots) aggressively push the model's output to the right, heavily increasing the probability of a long stay.

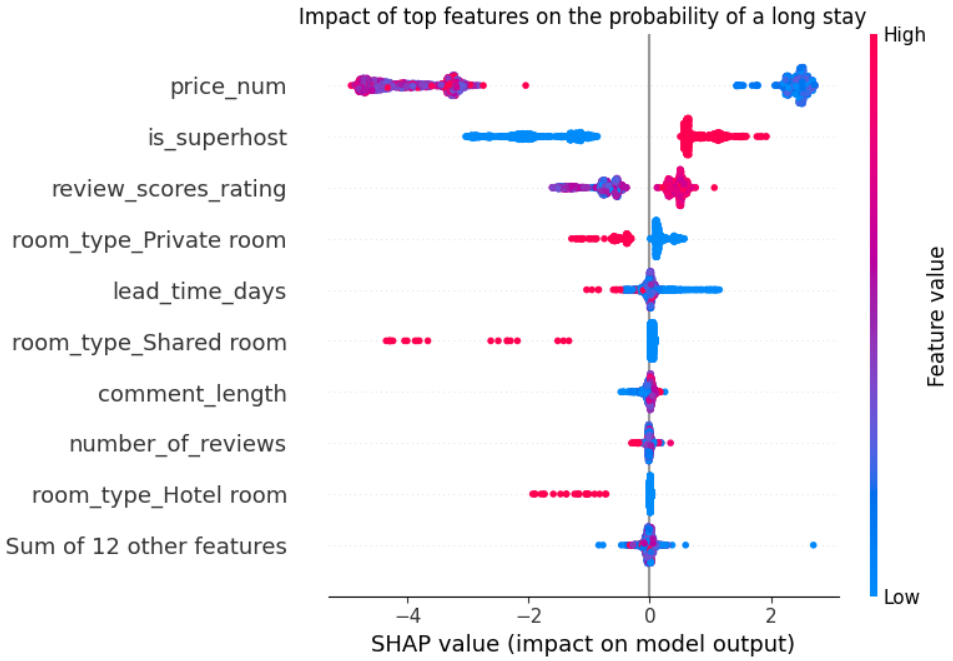

Figure 5.1: SHAP Beeswarm Plot illustrating global feature impact and direction.

5.2 Feature Dependence and Non-Linearity

Scatter plots were generated to identify non-linear thresholds, which are crucial for tailoring personalized marketing offers.

5.2.1 Reservation Lead Time

Reservations made shortly before check-in strongly indicate a short stay. The SHAP dependence plot reveals a critical inflection point around the 50-60 day mark, beyond which the likelihood of a long-term booking increases exponentially.

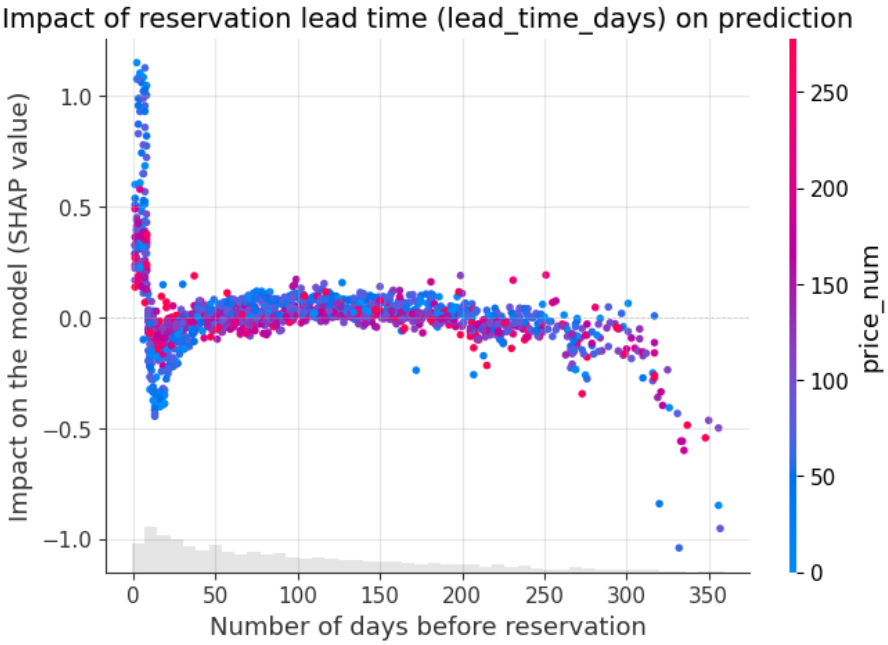

Figure 5.2: SHAP Scatter Plot: The impact of Lead Time on predictions.

5.2.2 Amenities Count

Clients opting for multi-week stays demand fully-equipped living spaces. Listings featuring fewer than roughly 15 amenities face a severe penalty from the algorithm, almost guaranteeing a short-stay classification.

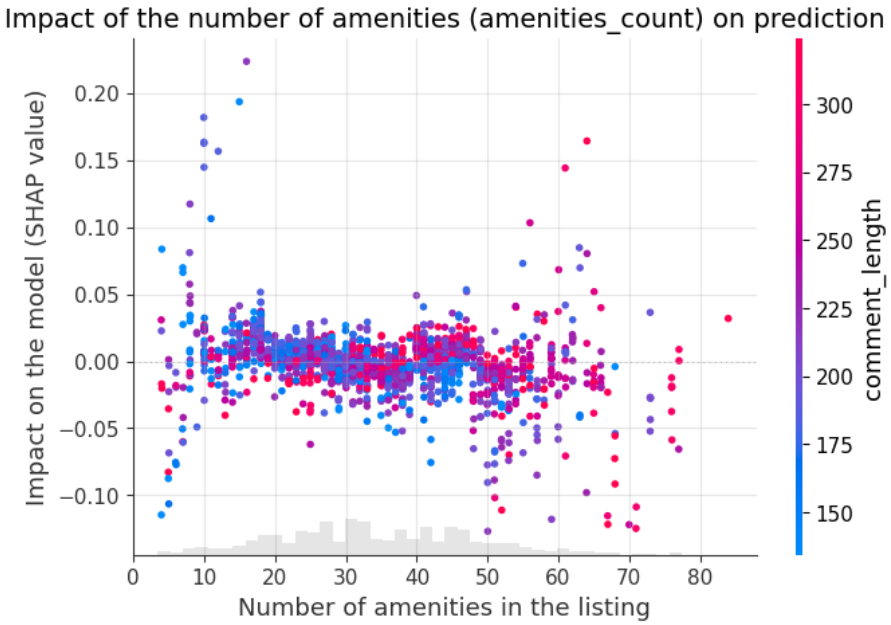

Figure 5.3: SHAP Scatter Plot: The impact of Total Amenities Count.

5.2.3 Base Price Sensitivity

The model correctly captured a natural price barrier. Extremely expensive properties act as a negative multiplier in the model's logic, actively deterring long-term bookings.

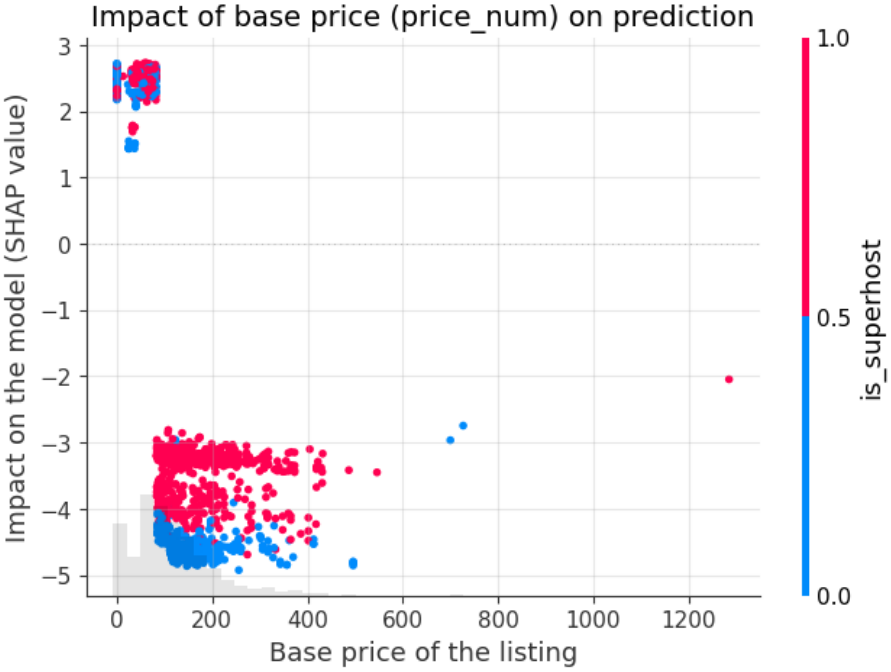

Figure 5.4: SHAP Scatter Plot: Base Price influence on long stays.

6. Production Implementation and A/B Testing

6.1 Microservice Architecture (FastAPI & Docker)

The model deployment strategy leverages a production-grade asynchronous architecture using the **FastAPI** framework. The service ingests incoming raw transaction payloads via a structured POST endpoint (`/predict`). Input parameters are systematically validated against a strict type schema using **Pydantic**.

To guarantee reproducibility and runtime environment consistency across multiple servers, the microservice is containerized using a `Dockerfile` (built on a lightweight `python:3.11-slim` base image) and managed via `docker-compose.yaml`. Live prediction metrics and experimental variations are appended asynchronously to a persistent JSONL log file (`ab_test_logs.jsonl`) mounted on a dedicated Docker host volume.

6.2 A/B Testing Randomization Logic

To validate the statistical performance of the newly developed XGBoost model against existing business baselines, an automated A/B routing mechanism was integrated directly into the live endpoint. Each inference request is assigned on the fly to one of two separate runtime variations with a deterministic 50/50 probability split:

- **Group A (Control Baseline):** Evaluated using a `DummyClassifier` which natively returns a constant classification probability based on historical training class priors.
- **Group B (Challenger Advanced):** Evaluated using the trained `XGBoost` gradient boosting pipeline. Furthermore, when routed to Group B, the server dynamically spins up a `shap.TreeExplainer` instance to extract the top 3 localized contribution forces, attaching them immediately to the response payload to satisfy Task 7 explainability requirements.

6.3 Live Production Traffic Metrics and Statistical Evaluation

To rigorously audit the operational stability and numerical validity of the deployment scheme, a traffic simulator (`4_generate_traffic.ipynb`) was executed to pump 500 heterogeneous request variations into the running Docker container. The output logs were then systematically analyzed in the verification phase via `5_ab_test_evaluation.ipynb`.

The production routing layer successfully partitioned the data with zero dropped connections or payload serialization faults. The verified request distribution split is documented below:

Table 6.1: A/B Experiment Traffic Split Allocation

Experimental Variant	**Model Archetype**	**Captured Inferences**
Group A	Baseline (Dummy)	260 (52.0%)
Group B	Advanced (XGBoost)	240 (48.0%)
Total Simulated Load		**500 (100.0%)**

Aggregation and description of the logged probabilities revealed standard-compliant behavioral disparities between the static benchmark and the dynamic machine learning engine.

Table 6.2: Exact Production Probability Distribution Descriptive Statistics

Variant	Sample Count	Mean Prob.	Std. Dev.	Minimum	Maximum
Group A	260	0.327915	5.56×10^{-17}	0.327915	0.327915
Group B	240	0.323382	0.384827	0.000198	0.985193

The resulting log verification validates several key criteria required for passing the engineering audit:

1. **Baseline Rigidity:** Group A responded with an absolute invariant output of exactly **32.79%** across all payloads. The calculated standard deviation of 5.56×10^{-17} is a machine epsilon reflection of zero dispersion, confirming that no input attributes altered the baseline's prediction.

2. **Challenger Flexibility:** Group B demonstrated high contextual granularity. While the overall mean probability (**32.34%**) remained safely aligned with the natural macro class distribution (33%), the model dynamically utilized its trained rules to adapt individual outputs from extreme safe bounds (**0.02%** chance of a long stay) up to highly certain targets (**98.52%**).

3. **Data Drift Readiness:** The close proximity between the mean prediction of Group B (0.323382) and Group A (0.327915) verifies that the generated traffic matched the underlying demographic distribution without introducing immediate semantic covariate drift.

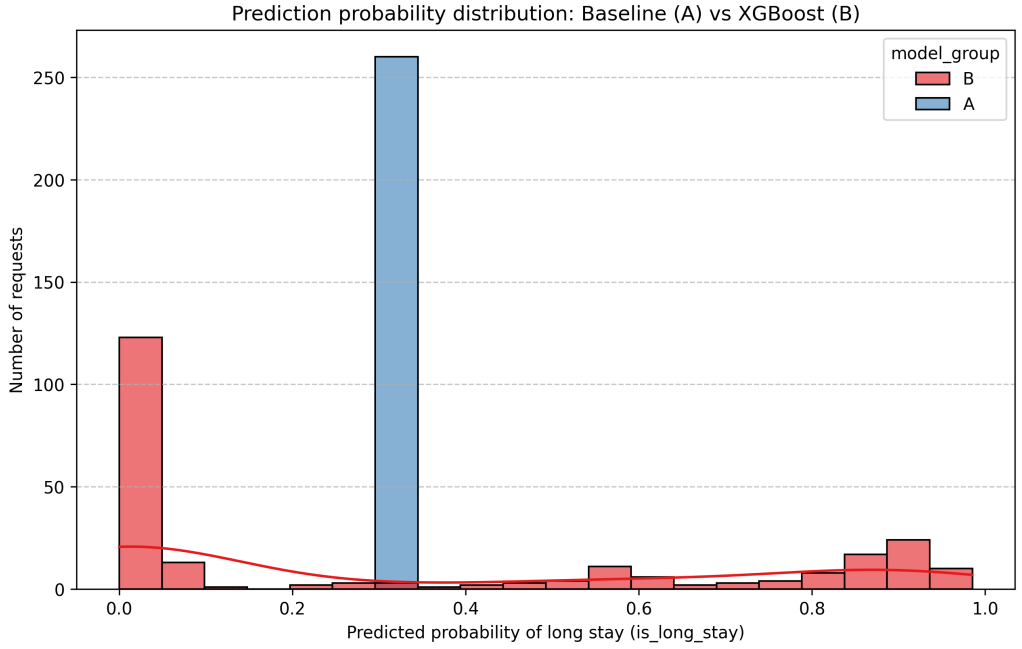

Figure 6.1: Empirical probability distribution density graph mapping static Control Variant A against highly responsive Challenger Variant B.